

Flujo, herramientas e instalación de L^AT_EX

Guía de referencia

Rodrigo Martín Vidal Galán

Junio de 2026

Documento de **referencia** de la suite: cómo **instalar** L^AT_EX, **compilar**, qué **herramientas** usar y cómo resolver las **problemáticas** típicas. A diferencia del *Quickstart* (que te lleva de la mano a tu primer PDF), acá explicamos *cómo funcionan* las piezas, para que entiendas qué pasa y puedas resolver cuando algo falla. No es un tutorial lineal: se consulta. (Para el universo de temas, ver *TEMARIO*; para el preámbulo recomendado, *Estándares y buenas prácticas*.)

Índice

1. Distribuciones de LaTeX	3
2. Editores e IDE	3
3. Compilar: motores y automatización	4
4. Instalar y actualizar paquetes	4
5. Problemáticas típicas (troubleshooting)	5
6. Diagnóstico y calidad	5
7. Control de versiones (git)	6
8. Reproducibilidad y CI	6
9. Velocidad (documentos grandes/lentos)	6
10.Recetas rápidas	6
Ver también	7

1. Distribuciones de LaTeX

L^AT_EX no es un solo programa, sino un **ecosistema**. Para usarlo instalás una **distribución**: un paquete grande que incluye el **motor** (el programa que compila), miles de **paquetes** de extensión (bajados del repositorio central, CTAN) y herramientas auxiliares (`latexmk`, `biber`, `texdoc`...). Sin una distribución instalada, no podés compilar nada.

Hay dos grandes distribuciones, y se diferencian sobre todo en la **filosofía de instalación de paquetes**; más una opción en la nube:

	MiKTeX	TeX Live	Overleaf
Plataforma	Windows (también Linux/-Mac)	Multiplataforma	Navegador (nube)
Instalación	Liviana; instala paquetes al vuelo	Completa (~varios GB, <code>scheme-full</code> o <code>scheme-basic</code>)	Ninguna
Gestor	mpm / MiKTeX Console	tlmgr	— (siempre al día)
En macOS	—	viene como MacTeX	—
Ideal para	Empezar en Windows	Setup completo/reproducible	Evitar instalar; colaborar

La diferencia clave. MiKTeX instala cada paquete *la primera vez* que un documento lo pide (cómodo y liviano, pero esa primera compilación puede frenarse unos segundos a descargar). TeX Live en su esquema completo instala *todo de entrada*: ocupa varios GB de disco, pero nunca te frena ni te falta nada —ideal para servidores, CI o trabajar sin conexión—. Overleaf elimina el problema de raíz: el entorno vive en la nube, siempre actualizado, y solo necesitás un navegador. **Recomendación:** en Windows para empezar, MiKTeX; para un setup completo o reproducible, TeX Live full; sin instalar nada o para trabajar en equipo, Overleaf.

2. Editores e IDE

Técnicamente podrías escribir el `.tex` en cualquier editor de texto y compilar por terminal. Pero un **editor de L^AT_EX** te ahorra muchísimo: resalta la sintaxis (ves de un vistazo comandos, llaves y errores de tipeo), compila con un **botón**, te muestra el **PDF al lado** y —gracias a SyncTeX— te deja saltar entre una línea del código y su lugar en el PDF (y al revés). Además marca los **errores** sobre la línea que los causa.

Editor	Notas
TeXstudio	Dedicado, «todo incluido», muy usado. Compilar: F5.
VS Code + LaTeX Workshop	Popular; integra compilación y <i>preview</i> .
Overleaf	Online, sin instalar; botón <i>Recompile</i> .
TeXmaker, Emacs/AUCTeX, Vim	Alternativas.

Aviso. LaTeX Workshop (VS Code) auto-compila al guardar. Eso es cómodo, pero deja procesos (`latexmk`, `pdflatex`) corriendo en segundo plano que a veces **bloquean archivos** y rompen las actualizaciones de paquetes; cómo resolverlo, en [Apartado 5](#).

3. Compilar: motores y automatización

Compilar **no siempre es un paso único**: L^AT_EX a veces necesita **varias pasadas**. Por ejemplo, para numerar referencias o armar el índice, la primera pasada «anota» los números en archivos auxiliares (`.aux`, `.toc`) y la segunda los coloca en su lugar. Por eso conviene **no** compilar a mano, sino con `latexmk`, que detecta cuántas pasadas hacen falta y corre también biber (bibliografía) y los índices.

El motor. Es el programa que efectivamente compila. `pdflatex` es el más rápido y compatible (el *default*). `xelatex` y `lualatex` permiten usar **fuentes del sistema** y Unicode nativo; `lualatex` además trae **Lua** para programar. Salvo que necesites eso, quedate con `pdflatex` (más en *Estándares*, sección de motor).

latexmk (el estándar). Resuelve solas todas las pasadas:

```
latexmk -pdf archivo.tex # compila a PDF (pdflatex)
latexmk -pdf -pvc archivo.tex # modo "watch": recompila al guardar
latexmk -xelatex archivo.tex # con otro motor
latexmk -outdir=build archivo.tex # intermedios en build/
latexmk -c # limpia intermedios (deja el PDF)
```

Para fijar opciones de forma permanente se usa un archivo `.latexmkrc`. Alternativas: `arara` (lee directivas escritas en el propio `.tex`) y `make`.

-shell-escape (con cuidado). Algunos paquetes (`minted` para resaltar código, `pythontex` para correr Python, `gnuplot`) necesitan que L^AT_EX **ejecute programas externos** durante la compilación; eso se habilita con `-shell-escape`. Como deja correr comandos del sistema, **no** compiles con `-shell-escape` documentos de origen desconocido (riesgo de seguridad).

4. Instalar y actualizar paquetes

Los paquetes viven en **CTAN** (el repositorio central, unos 6000); tu distribución los baja de ahí. **MiKTeX** puede instalarlos *al vuelo*: la primera vez que un `\usepackage` pide algo que no está, te ofrece bajarlo (aceptá). En **TeX Live** se instalan a mano con `tlmgr`. Conviene **actualizar** de vez en cuando: corrige errores y, sobre todo, mantiene los paquetes **compatibles entre sí** (mezclar versiones viejas y nuevas causa fallas; ver [Apartado 5](#)).

MiKTeX (manual):

```
mpm --install=NOMBRE
mpm --update=NOMBRE # (o MiKTeX Console > Updates)
initexmf --update-fndb # refrescar la base de nombres de archivo
```

TeX Live:

```
tlmgr install NOMBRE
tlmgr update --self --all # actualizar tlmgr y todo
```

Documentación offline de cualquier paquete (abre su manual oficial en PDF) —el mejor reflejo cuando no sabés cómo usar algo—:

```
texdoc amsmath
```

5. Problemáticas típicas (troubleshooting)

Tarde o temprano algo falla. La buena noticia: los errores de $\text{\LaTeX}$, aunque crípticos, son **repetitivos**. La estrategia es siempre la misma: **leer la primera línea** del error, ir al **número de línea** que indica, y aislar el problema. Los más comunes:

Síntoma	Causa	Solución
File 'X.sty' not found	Falta el paquete	MiKTeX lo ofrece instalar; o <code>tlmgr install X</code>
<code>\l__siunitx_..._str</code> undefined	<code>l3kernel/exp13</code> desfasado vs. un paquete nuevo	Actualizar el kernel (receta abajo)
Update falla: <i>file in use</i> / <i>Windows API error 32</i>	Un proceso TeX bloquea el <code>.fdb</code>	Matar los procesos TeX y reintentar
Fuentes borrosas / bitmaps	Falta <code>lmodern</code> (o <code>cm-super</code>)	<code>\usepackage{lmodern}</code>
Acentos rotos / «Invalid UTF-8»	Encoding / fuente	<code>inputenc[utf8] + fontenc[T1]</code>
Marcadores del PDF con símbolos raros	<code>hyperref</code> sin Unicode	<code>\hypersetup{unicode}</code>

Receta — kernel desfasado. Cuando `siunitx` (u otro paquete moderno) tira errores `\l__...`, casi siempre el `l3kernel/exp13` (la base de programación de $\text{\LaTeX}$) quedó viejo respecto del paquete. Se arregla actualizando el kernel (*incidente real de este proyecto*):

```
# Cerrar antes los procesos TeX que bloqueen archivos (Git Bash en Windows):
taskkill //F //IM latexmk.exe //IM pdflatex.exe //IM perl.exe
mpm --update=l3kernel --update=l3backend --update=l3packages
```

Receta — archivos bloqueados. En Windows, el auto-compilado del editor deja procesos TeX corriendo que **traban** el `.fdb` (la base de nombres de archivo de MiKTeX) y hacen fallar cualquier *update* con «file in use». Solución: matarlos antes (*también nos pasó*):

```
taskkill //F //IM latexmk.exe //IM pdflatex.exe //IM perl.exe
```

Cómo leer un error. Mirá la **primera línea** ! ... y el **número de línea**: casi siempre el problema está ahí (o una antes). Los errores que siguen suelen ser «efecto dominó» del primero. El `.log` guarda el detalle completo.

6. Diagnóstico y calidad

Además de los errores que **frenan** la compilación, $\text{\LaTeX}$ emite **advertencias** (*warnings*) que conviene atender: un `Overfull \hbox` significa que una línea se pasó del margen; **Reference undefined**, que una referencia no resolvió (¿compilaste dos veces?). El objetivo de un documento prolijo es compilar **sin warnings**.

- **nag** — te **avisa** en el log si usás comandos o paquetes obsoletos (te educa mientras escribís):
`\usepackage[l2tabu,orthodox]{nag}`
- **l2tabu** — la guía de referencia de **anti-patronés** (qué *no* usar y por qué).
- **Leer el log**: buscá `Overfull \hbox`, `Font shape undefined`, `Reference/Citation undefined`.

7. Control de versiones (git)

Como un `.tex` es **texto plano**, encaja perfecto con `git`: podés ver *exactamente* qué cambió entre versiones, volver atrás y colaborar sin pisarte. La clave es **no** versionar los archivos intermedios (`.aux`, `.log`...), que se regeneran al compilar; para eso está el `.gitignore`:

```
*.aux *.log *.out *.toc *.nav *.snm *.vrb
*.fls *.fdb_latexmk *.synctex.gz
*.bbl *.bcf *.run.xml # bibliografia
```

`latexdiff` va un paso más allá: compara dos versiones del `.tex` y genera un PDF con lo **agregado y borrado resaltado** (ideal para que un docente vea los cambios entre entregas):

```
latexdiff viejo.tex nuevo.tex > diff.tex && latexmk -pdf diff.tex
```

Para comentarios y marcas de revisión dentro del documento están `changes` y `todonotes`. **Overleaf** trae integración con `git` e historial propio.

8. Reproducibilidad y CI

¿Por qué importa? Para que el PDF salga **igual** en tu computadora, en la del docente y dentro de un año. Dos herramientas ayudan:

- **Docker**: una imagen `texlive/texlive` fija el entorno entero (misma versión de todo), así la compilación es idéntica en cualquier máquina.
- **GitHub Actions (CI)**: compila el PDF **automáticamente** en cada *push*; siempre hay un PDF actualizado sin que nadie compile a mano. Muy útil para entregables.

9. Velocidad (documentos grandes/lentos)

Un documento con muchos gráficos `TikZ` puede tardar **minutos**, porque cada figura se **redibuja** en cada compilación. Las técnicas de acá apuntan a **no recalcular lo que no cambió**:

- **Externalización de TikZ** (`\usetikzlibrary{external}`): cachea cada figura como un PDF; mientras no la toques, la reusa.
- **Modo draft**: no dibuja las imágenes (solo su recuadro) y marca los overfull; sirve para revisar texto rápido.
- **Preámbulo precompilado** (`mylatexformat`): si tu preámbulo es pesado, lo «congela» para no procesarlo cada vez.
- **latexmk -pvc**: no acelera la compilación, pero te da **feedback** inmediato recompilando al guardar.

10. Recetas rápidas

```
latexmk -pdf archivo.tex # compilar (recomendado)
latexmk -pdf -pvc archivo.tex # compilar y mirar cambios
latexmk -c # limpiar intermedios (deja el PDF)
latexmk -C # limpiar TODO (incluido el PDF)
texdoc PAQUETE # documentacion de un paquete
mpm --install=PAQUETE # instalar (MiKTeX)
tlmgr install PAQUETE # instalar (TeX Live)
```

Ver también

- *Estándares y buenas prácticas* — preámbulo recomendado y bases mínima/máxima.
- *Quickstart* — instalar y compilar tu primer documento.
- *README* y *TEMARIO* de la suite.